Studio Nx

JS 99 — markdownToFragment()

2026-10-13

JS 99 — `markdownToFragment()` (Markdown을 안전한 HTML로)

항목	내용
파일	`studionx/frontend/static/js/core/markdown.js` (새 파일), `static/vendor/marked/`, `static/vendor/dompurify/` (새 폴더)
작업	P1-23 화면 마무리: 관리자가 Markdown으로 쓴 공지 본문을 방문자 화면에 **안전하게** 보여 주는 변환 함수 (PRD 01 FR-114)
작성	Claude (AI), 2026-10-13, 사용자 검토
이전 · 다음	97 `loadDevNotes()` · 98 View More 모달 (다음. 번호는 순서와 달리 99를 먼저 만들었습니다: 보안이 걸린 이 함수를 먼저 시험하기 위해서입니다)

1. 하는 일

Markdown 글을 받아 **안전하게 걸러 낸 DOM 조각**(`DocumentFragment`)으로 돌려줍니다. 다음 함수(98번, 공지 모달)가 이 조각을 화면에 붙입니다. 사용자가 C안(라이브러리)을 골랐습니다 (2026-10-13, 목업: `docs/page_mockups/dev_notes_modal/`).

입력(Markdown)	결과
`# 제목`, `- 목록`, `**굵게**`, `` `코드` ``, `> 인용`, 표(`| a | b |`)	제목, 목록, 굵은 글자, 코드, 인용, 표 서식
`[글자](https://…)`	링크. **새 창으로 열리고** `rel="noopener noreferrer"`가 붙습니다
줄바꿈(Enter 한 번)	줄바꿈(` `)
`<script>`, `<iframe>`, `<form>`, `<style>`, `<svg>`, `<object>`	지워짐
`![그림](주소)`, `<img>`	지워짐 (외부 그림은 방문 기록이 새는 길이라 막습니다)
`onclick`, `onerror`, `ontoggle` 같은 `on…` 속성, `style` 속성	지워짐
`javascript:`, `data:` 주소의 링크, 상대 경로와 `#` 링크	주소(`href`)가 지워져 그냥 글자만 남음. `http`, `https`, `mailto`만 링크로 남습니다
글이 아닌 값(`null`, 숫자, 빈 글자)	빈 조각 (오류 없음)

2. 새로 들어온 도구와 SweetAlert2

도구	버전, 라이선스	위치	하는 일
marked	18.1.1, MIT	`static/vendor/marked/marked.esm.js`	Markdown → HTML
DOMPurify	3.4.17, MPL-2.0 또는 Apache-2.0	`static/vendor/dompurify/purify.es.mjs`	위험한 HTML 제거

- SweetAlert2(FIX-01)와 **같은 방식**입니다. npm 패키지의 ES 모듈 파일을 `static/vendor/`에 넣었고, 외부 CDN은 쓰지 않습니다. 라이선스 파일과 버전을 적은 `VERSION.txt`를 함께 두었습니다. 원본에서 바꾼 것은 파일 끝의 `sourceMappingURL` 주석 한 줄을 뺀 것뿐입니다 (브라우저가 없는 `.map` 파일을 찾느라 404를 내지 않게 하려는 것입니다).

- **SweetAlert2에는 영향이 없습니다.** 새 도구는 ES 모듈이라 전역 변수를 만들지 않고, 다른 폴더(`vendor/marked`, `vendor/dompurify`)에 있어 `vendor/sweetalert2` 파일은 그대로입니다. 시험 S01 ~ S03에서 확인했습니다 (§6).

3. 언제 쓰나

호출하는 곳	시점
98 공지 모달 (다음)	공지 상세에서 본문(`content`)을 보여 줄 때

4. 코드

```
import { marked } from '../../vendor/marked/marked.esm.js';
import DOMPurify from '../../vendor/dompurify/purify.es.mjs';

export function markdownToFragment(markdown) {
  if (typeof markdown !== 'string' || markdown === '') return document.createDocumentFragment();

  const html = marked.parse(markdown, { gfm: true, breaks: true, async: false });
  const fragment = DOMPurify.sanitize(html, {
    RETURN_DOM_FRAGMENT: true,
    USE_PROFILES: { html: true },
    FORBID_TAGS: ['style', 'form', 'input', 'button', 'textarea', 'select', 'option', 'img', 'picture',
      'source', 'video', 'audio', 'iframe', 'object', 'embed', 'link', 'meta', 'base'],
    FORBID_ATTR: ['style', 'srcset'],
    ALLOWED_URI_REGEXP: /^(?:https?:|mailto:)/i,
    ALLOW_DATA_ATTR: false,
  });

  const links = fragment.querySelectorAll('a');
  for (const link of links) {
    if (!link.hasAttribute('href')) continue;
    link.setAttribute('target', '_blank');
    link.setAttribute('rel', 'noopener noreferrer');
  }
  return fragment;
}
```

최신 문법으로 쓰면 (공부용이며, 실제 코드는 위에 있습니다)

```
export const markdownToFragment = (markdown) => {
  if (typeof markdown !== 'string' || !markdown) return document.createDocumentFragment();
  const fragment = DOMPurify.sanitize(marked.parse(markdown, OPTIONS), PURIFY_OPTIONS);
  fragment.querySelectorAll('a[href]').forEach((a) => a.setAttribute('target', '_blank') || a.setAttribu
te('rel', 'noopener noreferrer'));
  return fragment;
};
```

최신 문법	실제 코드	뜻
`querySelectorAll('a[href]')`	`querySelectorAll('a')`와 `hasAttribute('href')` 검사	`href`가 있는 링크만 고릅니다
`.forEach((a) => …)`	`for (const link of links)`	목록을 하나씩 처리합니다

최신 문법	실제 코드	뜻
`!markdown`	`markdown === ''`	빈 글자를 거짓으로 봅니다

문법 풀이

식	뜻
`marked.parse(글, { gfm: true, breaks: true, async: false })`	Markdown을 HTML 글자로 바꿉니다. `gfm`은 표 같은 확장 문법, `breaks`는 Enter 한 번도 줄바꿈, `async: false`는 결과를 바로 돌려받는다는 뜻입니다
`DOMPurify.sanitize(html, 설정)`	HTML에서 위험한 부분을 지웁니다
`RETURN_DOM_FRAGMENT: true`	글자가 아니라 DOM 조각으로 돌려줍니다. 글자(`innerHTML`)로 다시 바꿔서 넣으면 브라우저가 한 번 더 해석해서 우회될 수 있어서(변이 XSS) 조각 그대로 붙입니다
`USE_PROFILES: { html: true }`	일반 HTML 태그만 허용하는 기본 묶음입니다 (SVG, MathML 제외)
`FORBID_TAGS`, `FORBID_ATTR`	기본 허용 범위에서 추가로 막을 태그와 속성입니다
`ALLOWED_URI_REGEXP: /^(?:https?:|mailto:)/i`	링크 주소가 `http:`, `https:`, `mailto:`로 시작할 때만 남깁니다 (`i`는 대소문자 무시)
`ALLOW_DATA_ATTR: false`	`data-…` 속성을 막습니다
`document.createDocumentFragment()`	비어 있는 DOM 조각입니다
`link.hasAttribute('href')`	주소가 있는 링크인지 봅니다. 위험한 주소라서 지워졌으면 `href`가 없습니다

5. 흐름

1. 글이 아니거나 비어 있으면 빈 조각을 돌려줍니다.
2. `marked`가 Markdown을 HTML로 바꿉니다.
3. `DOMPurify`가 그 HTML을 걸러서 DOM 조각으로 만듭니다.
4. 남은 링크마다 새 창으로 열고 `rel`을 붙입니다.
5. 조각을 돌려줍니다. 부르는 쪽은 `appendChild`로 붙이면 됩니다.

6. 왜 이렇게

선택	이유
변환 → 안전 처리 → 링크 손질을 **한 함수에 고정**	안전 처리를 부르는 쪽이 빠뜨릴 수 없습니다. 본문을 HTML로 넣는 곳은 이 함수 하나뿐이어야 합니다
허용 목록 방식(필요한 것만 남김)	막을 것을 나열하면 빠뜨린 것이 구멍이 됩니다. DOMPurify의 기본 허용 범위에서 필요 없는 것을 더 줄였습니다
그림(`img`, `svg`, `video`)도 막음	공지에 그림이 필요하면 나중에 허용 주소를 정해 열 수 있습니다. 지금은 외부 주소로 방문자의 접속 기록이 새는 것을 막는 쪽을 택했습니다
`style` 속성과 태그를 막음	글이 화면 전체를 가리거나(`position: fixed`) 숨기는(`display: none`) 장난을 막습니다
링크 주소를 `http`, `https`, `mailto`로 제한	`javascript:`, `data:` 주소가 눌렀을 때 실행되는 것을 막습니다
새 창 + `noopener noreferrer`	열린 사이트가 우리 창을 조작하지 못하게 하고, 우리 주소를 넘겨주지 않습니다
`breaks: true`	관리자는 입력칸에서 Enter로 줄을 나눕니다. Markdown 규칙(빈 줄이 있어야 문단)을 몰라도 쓴 대로 보입니다

선택	이유
조각(`DocumentFragment`)으로 돌려줌	글자로 돌려주면 `innerHTML`로 넣게 되는데, 걸러 낸 결과를 다시 글자로 만들었다가 해석하는 과정에서 우회가 가능합니다 (변이 XSS)
글 안의 `#` 제목은 태그를 바꾸지 않고 CSS로 크기를 맞춤	모달 제목과 공지 제목보다 크게 보이지 않게 합니다 (98번에서 CSS)

7. 시험 (실제 서버, 헤드리스 Chrome, 26개)

브라우저에서 변환한 결과를 **실제 문서에 붙여서**(실행될 수 있는 상황) 확인했습니다. 공격 글이 실행되면 `window.__xss`가 1로 바뀌도록 해 두었습니다.

묶음	시험	결과
서식	제목, 목록, 굵게, 링크, 코드, 인용, 표가 모두 서식으로 바뀜	✓
링크	새 창, `rel="noopener noreferrer"`	✓
줄바꿈	Enter 한 번이 ` `	✓
공격 글 16종	`<script>`, `<img onerror>`, 마크다운 그림, `javascript:` 링크, 대소문자를 섞은 `JaVaScRiPt:`, 줄바꿈으로 깨뜨린 `java
script:`, `data:` 링크, HTML `<a href="javascript:…" onclick>`, `<svg onload>`, `<iframe>`, `<style>`와 `style` 속성, `<form>`과 입력칸, `<details ontoggle>`, `<math>` 변이 XSS, 링크 글자 안의 `<img onerror>`, `<object>`와 `<embed>`	모두 실행되지 않음 (`__xss`가 0), `on…` 속성 0개, 위험한 태그와 주소가 남지 않음 ✓
일반	안전한 글자는 남고 `<script>`만 사라짐	✓
일반	`mailto:`, `http://`만 링크로 남고 상대 경로와 `#` 링크는 `href`가 지워짐	✓
이상한 입력	`null`, `undefined`, 숫자, 객체, 배열, 빈 글자를 넣으면 빈 조각	✓
성능	3000줄(링크 3000개)을 1초 안에 처리	✓
SweetAlert2	두 도구를 불러온 뒤에도 **새 전역 변수가 생기지 않음** (`window.marked`, `window.DOMPurify` 없음)	✓
SweetAlert2	`showAlert`로 알림창이 그대로 뜸	✓
SweetAlert2	`vendor/sweetalert2` 파일이 그대로 제공됨 (200)	✓

26개 모두 통과했습니다. 처음 돌렸을 때 5개(`javascript:`, `data:` 링크 등)가 실패로 나왔는데, 위험한 주소의 `href`가 **지워지는 것이 의도된 결과**인데 시험이 "`href`가 있어야 한다"고 잘못 확인한 시험의 문제였습니다. 이때 함수도 하나 고쳤습니다. `href`가 지워진 링크에도 `target`과 `rel`이 붙고 있었는데, 주소가 없는 링크는 글자로만 남도록 건너뛰게 했습니다.